ZAP! Compiler Error Messages

The author of this software stands in solidarity with 🇺🇦 Ukraine. We believe in a world where international borders are respected and human rights are upheld. We encourage all users of this software to contribute to humanitarian efforts in 🇺🇦 Ukraine.

This guide explains the most common error messages you may encounter when compiling ZAP! programs, with examples and solutions.

Reading Error Messages

All errors follow the format:

```
filename:line:column: error: message
```

For example:

```
main.zap:15:5: error: Undefined variable 'count'
```

This tells you:

- **File**: main.zap
- **Line**: 15
- **Column**: 5
- **Message**: The variable `count` was used but never declared

Tokenizer Errors

These occur when the compiler cannot understand the raw text of your source file.

Unknown character

```
error: Unknown character: @
```

Cause: A character that is not part of ZAP! syntax. **Fix**: Check for typos or unsupported characters.

Missing end of string

```
error: Missing end of string
```

Cause: A string literal was opened with `"` but never closed. **Fix**: Add the closing `"` on the same line.

Identifier starting with underscore

```
error: '_' is not allowed as first character
```

Cause: Variable or identifier starts with `_`. Names starting with underscore are reserved for compiler-generated symbols. **Fix**: Rename to start with a letter.

Invalid escape sequence

```
error: Unknown escape sequence: \q
```

Cause: Unrecognized escape in a string or character literal. **Fix**: Valid escapes: `\n`, `\t`, `\0`, `\\`, `\"`, `\'`, `\xHH`, `\OOO`, `\bBBBBBBBB`.

Preprocessor Errors

These occur during the `.define` / `.ifdef` preprocessing pass.

Symbol already defined

```
error: Symbol 'ATARI' already defined
```

Cause: `.define ATARI` used when `ATARI` is already defined (possibly via `-D` flag). **Fix**: Use `.ifndef ATARI` / `.define ATARI` / `.endif` to guard the definition.

Unclosed conditional

```
error: Unclosed .ifdef/.ifndef (missing .endif)
```

Cause: An `.ifdef` or `.ifndef` block was opened but never closed. **Fix**: Add `.endif` to close the conditional block.

Unexpected .else / .endif

```
error: Unexpected .else without .ifdef/.ifndef
error: Unexpected .endif without .ifdef/.ifndef
```

Cause: `.else` or `.endif` appears without a matching `.ifdef`/`.ifndef`. **Fix**: Check that your conditional blocks are properly nested.

Syntax Errors

These occur when the parser encounters unexpected tokens.

Expected declaration

```
error: Expected declaration, PROC, FUNC, or STRUCT
```

Cause: The parser found something at the top level that is not a valid declaration, procedure, function, or struct definition. **Fix**: Check for misplaced statements outside of procedures/functions.

Unknown type

```
error: Unknown type 'integer'
```

Cause: A type name was used that is not `byte`, `word`, `long`, or a defined struct. **Fix**: Use one of the built-in types or define a struct first.

Local declarations must precede statements

```
error: Local variable declarations must be placed before the first
statement in a procedure
```

Cause: A variable declaration appears after executable statements inside a procedure or function. **Fix**: Move all local variable declarations to the top of the procedure, before any executable code.

```
proc example()
    byte x        ; OK: declaration first
    x = 10        ; OK: statement after declarations
    byte y        ; ERROR: declaration after statement
end
```

Duplicate parameter / local

```
error: Duplicate parameter 'value' in procedure
error: Duplicate local 'count' in procedure
```

Cause: Two parameters or locals with the same name. **Fix**: Rename one of them.

Missing UNTIL

```
error: Expected UNTIL to close REPEAT block
```

Cause: A `repeat` block is missing its `until` condition. **Fix**: Add `until expression` at the end of the repeat block.

Semantic Errors (Types and Declarations)

These occur after parsing when the compiler checks meaning and correctness.

Undefined variable / procedure / function

```
error: Undefined variable 'count'
error: Undefined procedure 'init'
error: Undefined function 'calculate'
```

Cause: Using a name that has not been declared. **Fix**: Declare it before use, or check for typos (names are case-insensitive).

Variable already defined

```
error: Variable 'x' already defined
```

Cause: Declaring a variable with a name that is already in use. **Fix**: Use a different name.

Program must have main()

```
error: Program must have a 'main()' procedure
```

Cause: No `proc main()` found in the program. **Fix**: Every ZAP! program needs a `proc main() ... end` as the entry point.

CONST must have initializer

```
error: CONST must have initializer (expression, list, or string)
```

Cause: A `const` variable declared without a value. **Fix**: Provide an initial value: `const byte MAX = 100`.

STATIC and CONST cannot be combined

```
error: STATIC and CONST modifiers cannot be combined
```

Cause: Using both `static` and `const` on the same variable. **Fix**: Use one or the other. `const` is compile-time; `static` is runtime-persistent.

PORT modifier restrictions

```
error: PORT modifier requires address specification with @
error: PORT modifier cannot be used on arrays
error: PORT variable cannot have initializer
```

Cause: `#PORT` variables must have a fixed address and cannot be arrays or initialized. **Fix**: Declare as `byte REG @$D000 #PORT`.

Semantic Errors (Expressions)

Cannot dereference non-pointer

```
error: Cannot dereference non-pointer
```

Cause: Using `^` on a value that is not a pointer. **Fix**: Ensure the variable is declared as a pointer type (e.g., `byte ^ptr`).

Cannot add two pointers

```
error: Cannot add two pointers
```

Cause: Adding two pointer values together (e.g., `ptr1 + ptr2`). **Fix**: Add a numeric offset to a pointer instead: `ptr + 5`.

Division by zero

```
error: Division by zero
error: Division by zero in constant expression
```

Cause: Dividing by zero in a constant expression or detected at compile time. **Fix**: Check divisor values.

Array index out of bounds

```
error: Array index 10 is out of bounds for array dimension 1 with size 10
```

Cause: A constant array index exceeds the array size (indices are 0-based). **Fix**: For `byte arr[10]`, valid indices are 0 through 9.

Array index cannot be negative

```
error: Array index cannot be negative: -1
```

Cause: A constant array index is negative (e.g., `arr[-1]`). **Fix**: Array indices must be 0 or greater. Use a runtime variable if negative offsets are intended via pointer arithmetic.

Fixed variable address cannot be negative

```
error: Fixed variable address cannot be negative: -1
```

Cause: The `@address` in a fixed-address declaration evaluates to a negative constant (e.g., `byte x @-1`). **Fix**: Provide a valid memory address (0–65535).

Read from write-only port

```
error: Read from write-only port
```

Cause: Reading a variable declared with `#WR` (write-only). **Fix**: Use `#RD` or `#PORT` (both read and write) if the port supports reading.

Struct values in binary expressions

```
error: Struct values cannot be used in binary expressions
```

Cause: Using a struct in arithmetic (e.g., `myStruct + 1`). **Fix**: Access individual fields: `myStruct.x + 1`.

Semantic Errors (Functions and Procedures)

Wrong number of arguments

```
error: Procedure 'draw' expects 3 parameter(s), but 2 were/was provided
error: Function 'add' expects 2 parameter(s), but 4 were/was provided
```

Cause: Calling with the wrong number of arguments. **Fix**: Match the parameter count. Use `,` for skipped default parameters: `draw(1, , 3)`.

Argument width mismatch

```
error: Argument 1 of 'draw': cannot pass WORD to BYTE parameter, use LOW()
or HIGH()
error: Argument 2 of 'calc': cannot pass LONG to BYTE parameter, use
LOW()/HIGH()/LOWW()/HIGHW()
error: Argument 1 of 'move': cannot pass LONG to WORD parameter, use LOWW()
or HIGHW()
```

Cause: Passing a wider type to a narrower parameter would silently truncate data. **Fix**: Use explicit narrowing functions: `LOW()`, `HIGH()` for WORD-to-BYTE; `LOWW()`, `HIGHW()` for LONG-to-WORD. Constants that fit in the parameter type are allowed (e.g., `foo(42)` is valid for a BYTE parameter).

PEEK/POKE type errors

```
error: PEEK() address must be BYTE or WORD, use LOWW() or HIGHW() for LONG
values
error: POKE() value must be BYTE, use LOW() or HIGH() for WORD values
error: POKE() value must be BYTE, use LOW()/HIGH()/LOWW()/HIGHW() for LONG
values
```

Cause: PEEK/POKE operate on single bytes. Address must fit in 16 bits; value must be 8 bits. **Fix**: Use narrowing functions as suggested in the error message.

Return type mismatch

```
error: RETURN type mismatch: expected BYTE, got WORD
```

Cause: Returning a value that does not match the function's declared return type. **Fix**: Ensure the return expression matches the function signature.

Function must have RETURN

```
error: FUNC must have RETURN
```

Cause: A function body does not end with a `return` statement. **Fix**: Add `return expression` at the end of every function.

Semantic Errors (Assignment)

Cannot assign to const

```
error: Cannot assign to const variable 'MAX'
error: Cannot assign to field of const struct 'config'
error: Cannot assign to element of const array 'table'
```

Cause: Attempting to modify a `const` value at runtime. **Fix**: Remove the `const` modifier if the value needs to change.

Left side not assignable

```
error: Left side of assignment is not assignable
```

Cause: The left side of `=` is not a valid target (not a variable, array element, struct field, or pointer dereference). **Fix**: Ensure the left side is a writable l-value.

Code Generation Errors

Math stack overflow

```
error: Math stack overflow: requires more than 8 slots. Simplify the
expression.
```

Cause: An arithmetic expression is too deeply nested for the compiler's evaluation stack. **Fix**: Break the expression into smaller parts using temporary variables.

Expression too complex

```
error: Expression too complex: requires N temporary slot(s) but hard limit
is M. Simplify the expression.
```

Cause: Similar to stack overflow — the expression needs more temporary storage than available. **Fix**: Split into simpler sub-expressions.

CLI Errors

ZPSTART value out of range

```
error: -ZPSTART value must be 0-255
```

Cause: The zero-page start address is outside the valid 8-bit range. **Fix**: Use a value between 0 and 255.

Configuration file errors

```
error: Config file 'linker.cfg' not found
error: No ZP segment start found in config file
```

Cause: The `-cfg` flag points to a missing or invalid ld65 linker configuration file. **Fix**: Verify the file path and ensure it contains a ZEROPAGE memory segment.

Tips for Debugging

1. **Read the line number** — The error always points to the exact location.
2. **Check the line above** — Sometimes the real problem is on the previous line (e.g., missing `end`).
3. **Case does not matter** — ZAP! is case-insensitive for identifiers, so `myVar` and `MYVAR` are the same.
4. **Declare before use** — All variables must be declared before their first use.
5. **Declarations first** — Inside procedures/functions, all local variable declarations must come before any statements.
6. **Use the IDE** — The VS Code extension shows errors inline with 1.5-second delay after typing.